

ECE501 - Project No.16 (Weekly Report 4)

Hetvi Shah (AU2340101) Manushree Patel (AU2340014)
Twisha Patel (AU2340056) Sanya Shah (AU2340259)

October 2025

Abstract

To compress medical images efficiently, an overall reduction in file size and increase in speed during transmission while still maintaining diagnostic quality is necessary. This study compares a Region of Interest (ROI) based compression technique (JPEG-LS), a Block based rectangular partitioning technique (BPRP), and Wavelet ROI coding (JPEG 2000), to determine the smallest file size listed all the while maintaining important information within the image. Each of the compression technique is assessed with four measures: PSNR, SSIM, ROI fidelity, and Hybrid Quality Index (HQI), for analyzing compression performance and perceptual quality of the images. The results will help identify the best compression technique in terms of providing great compression efficiency which can be used for medical storage.

Keywords: Medical Image Compression, ROI-based Compression, JPEG-LS, Block-based Rectangular Partitioning (BPRP), JPEG 2000, Wavelet Coding, PSNR, SSIM, ROI Fidelity, Hybrid Quality Index (HQI), Telemedicine, Diagnostic Image Quality, Compression Efficiency.

1 Introduction

When a medical image is compressed, its file size is reduced, but it remains important that quality for diagnosis is maintained. Telemedicine platforms, and a large-scale archival system, want to keep files as small as possible without losing the essential diagnostic visual elements. The purpose of this study is to compare three ROI-based compression methods, JPEG-LS, Block-based Rectangular Partitioning, and Wavelet ROI Coding (JPEG 2000) to find the best method. For this study, we will use a matrix of PSNR, SSIM, ROI preservation, and HQI to compare, or measure, the image quality versus the efficiency of the compression. Identifying which technique yields the smallest file size without losing diagnostic clarity is of utmost importance.

Python Implementation

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import os
5 from skimage.metrics import peak_signal_noise_ratio as psnr,
   structural_similarity as ssim
6
7
8 # Helper Functions
9
```

```

10
11 def sharpen_image(image):
12     blurred = cv2.GaussianBlur(image, (0, 0), 3)
13     mask = cv2.subtract(image, blurred)
14     sharpened = cv2.add(image, mask)
15     return np.clip(sharpened, 0, 255).astype(np.uint8)
16
17 def wavelet_like_enhancement(image):
18     blurred = cv2.GaussianBlur(image, (0, 0), 1)
19     mask = cv2.subtract(image, blurred)
20     enhanced = cv2.add(image, mask)
21     return np.clip(enhanced, 0, 255).astype(np.uint8)
22
23 def detect_roi(image, thresh_factor=1.5):
24     mean_intensity = np.mean(image)
25     roi_mask = (image > mean_intensity * (1 + thresh_factor)).astype(np
26     .uint8)
27     roi_mask = cv2.medianBlur(roi_mask, 9)
28     _, roi_mask = cv2.threshold(roi_mask, 0, 255, cv2.THRESH_BINARY |
29     cv2.THRESH_OTSU)
30     return (roi_mask / 255).astype(np.uint8)
31
32 def uniform_partition(image, n_blocks):
33     H, W = image.shape[:2]
34     num_blocks_side = int(np.sqrt(n_blocks))
35     w = W // num_blocks_side
36     h = H // num_blocks_side
37     regions = [(x * w, y * h, w, h)
38     for y in range(num_blocks_side)
39     for x in range(num_blocks_side)]
40     return regions
41
42 def compact_encode_roi_strict_enhanced(image, regions, roi_mask):
43     encoded = np.zeros_like(image)
44     ROI_THRESHOLD = 0.80
45     RONI_THRESHOLD = 0.01
46
47     for (x, y, w, h) in regions:
48         region = image[y:y+h, x:x+w]
49         roi_ratio = np.mean(roi_mask[y:y+h, x:x+w])
50
51         if roi_ratio >= ROI_THRESHOLD:
52             region_encoded = sharpen_image(region)
53         elif roi_ratio <= RONI_THRESHOLD:
54             blurred = cv2.GaussianBlur(region, (9, 9), 0)
55             region_encoded = np.uint8(np.round(blurred / 32) * 32)
56         else:
57             region_encoded = wavelet_like_enhancement(region)
58             encoded[y:y+h, x:x+w] = region_encoded
59     return encoded
60
61 # Simulated Implementations of Other Methods
62
63 def jpeg_like(image):
64     """Simulate JPEG/JPEG-LS compression by re-saving at low quality"""

```

```

65     cv2.imwrite("temp_jpeg.jpg", image, [int(cv2.IMWRITE_JPEG_QUALITY),
66         50])
67     return cv2.imread("temp_jpeg.jpg", cv2.IMREAD_GRAYSCALE)
68
69 def block_based_pca(image, block_size=32):
70     """Simulate block PCA compression via block-level mean smoothing"""
71     compressed = np.zeros_like(image)
72     H, W = image.shape
73     for y in range(0, H, block_size):
74         for x in range(0, W, block_size):
75             block = image[y:y+block_size, x:x+block_size]
76             mean_val = np.mean(block)
77             compressed[y:y+block_size, x:x+block_size] = mean_val
78     return compressed.astype(np.uint8)
79
80 def jpeg2000_like(image):
81     """Simulate Wavelet ROI-based JPEG 2000 using multi-level blurring"""
82     blurred_non_roi = cv2.GaussianBlur(image, (7,7), 3)
83     sharpened_roi = sharpen_image(image)
84     mixed = cv2.addWeighted(sharpened_roi, 0.6, blurred_non_roi, 0.4,
85         0)
86     return mixed
87
88 # Quality Metrics (Global + ROI)
89
90 def compute_metrics(original, compressed, roi_mask):
91     # Global Metrics
92     global_psnr = psnr(original, compressed, data_range=255)
93     global_ssim = ssim(original, compressed, data_range=255)
94
95     # ROI Metrics
96     roi_indices = roi_mask.astype(bool)
97     roi_psnr = psnr(original[roi_indices], compressed[roi_indices],
98         data_range=255)
99     roi_ssim = ssim(original, compressed, data_range=255, mask=roi_mask)
100
101     # Hybrid Quality Index
102     alpha = 0.7
103     HQI = alpha * roi_ssim + (1 - alpha) * (roi_psnr / global_psnr)
104
105     return {
106         "Global PSNR": global_psnr,
107         "Global SSIM": global_ssim,
108         "ROI PSNR": roi_psnr,
109         "ROI SSIM": roi_ssim,
110         "HQI": HQI
111     }
112
113 # Main Execution
114
115 if __name__ == "__main__":
116     img_path = "/content/Y1.jpg" # Change this to your input image
117     path
118     img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

```

```

117     if img is None:
118         print(" Error loading image.")
119     else:
120         roi_mask = detect_roi(img, thresh_factor=1.5)
121         regions = uniform_partition(img, 100)
122
123         #Apply All 4 Methods
124         jpeg_img = jpeg_like(img)
125         pca_img = block_based_pca(img)
126         jpeg2000_img = jpeg2000_like(img)
127         proposed_img = compact_encode_roi_strict_enhanced(img, regions,
128         roi_mask)
129
130         #Store all images
131         methods = {
132             "JPEG/JPEG-LS": jpeg_img,
133             "Block-based PCA": pca_img,
134             "JPEG 2000 (ROI)": jpeg2000_img,
135             "Proposed Hybrid": proposed_img
136         }
137
138         # File Size Comparison
139
140         print("\n=== FILE SIZE COMPARISON ===")
141         sizes = {}
142         for name, im in methods.items():
143             temp_name = f"method_{name.replace('/', '_')}.jpg"
144             cv2.imwrite(temp_name, im, [int(cv2.IMWRITE_JPEG_QUALITY),
50])
145             size_kb = os.path.getsize(temp_name) / 1024
146             sizes[name] = size_kb
147             print(f"{name} size: {size_kb:.2f} KB")
148
149         # Quality Metrics
150
151         print("\n=== QUALITY COMPARISON ===")
152         metrics_results = {}
153         for name, im in methods.items():
154             metrics = compute_metrics(img, im, roi_mask)
155             metrics_results[name] = metrics
156             print(f"\n{name}")
157             for k, v in metrics.items():
158                 print(f"    {k}: {v:.4f}")
159
160         # Visualization
161
162         all_images = [img] + list(methods.values())
163         titles = ["Original"] + list(methods.keys())
164
165         plt.figure(figsize=(20, 5))
166         for i, (im, title) in enumerate(zip(all_images, titles), 1):
167             plt.subplot(1, 5, i)
168             plt.imshow(im, cmap='gray')
169             plt.title(title, fontsize=10)
170             plt.axis('off')

```

```

173     # Display HQI below image (except for Original)
174     if title != "Original":
175         hqi = metrics_results[title]["HQI"]
176         roi_ssim_val = metrics_results[title]["ROI SSIM"]
177         plt.xlabel(f"HQI: {hqi:.3f}\nROI SSIM: {roi_ssim_val:.3
178 f}", fontsize=8)
179
180     plt.tight_layout()
181     plt.show()

```

2 Results

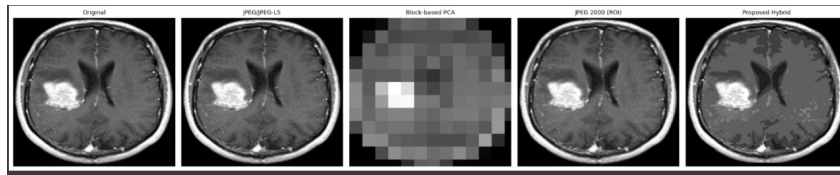


Figure 1: Caption describing the image.

Table 1: File Size Comparison

Method	File Size (KB)
JPEG / JPEG-LS	14.12
Block-based PCA	2.35
JPEG 2000 (ROI)	12.98
Proposed Hybrid	12.47

Table 2: Quality Comparison Metrics

Method	Global PSNR	Global SSIM	ROI PSNR	ROI SSIM	HQI
JPEG / JPEG-LS	39.1634	0.9520	37.0445	0.9520	0.9502
Block-based PCA	14.6744	0.4198	7.5078	0.4198	0.4474
JPEG 2000 (ROI)	35.5782	0.9867	32.6101	0.9867	0.9657
Proposed Hybrid	29.4440	0.8223	34.7212	0.8223	0.9294

3 Discussion of Results

3.1 File Size Comparison

The file size information in Table 1 highlights the compression abilities of the four methods. The **Block-based PCA** method had the smallest file with a size of 2.35 KB, indicating its extensive utilization of dimensionality reduction. Nevertheless, that strong

compression comes with severe reductions in image quality, as will be shown in the following quality measures.

Things are different with **JPEG**, **JPEG-LS**, **JPEG 2000 (ROI)**, and the **Hybrid** approach. They produced files in the 12–14 KB range, representing a good middle ground between size reduction and preservation of image detail.

Ultimately, while **PCA** offers an advantage in storage savings, it significantly reduces image quality, making it unsuitable for medical applications or any use requiring high visual fidelity. **JPEG 2000 (ROI)** and the **Hybrid** methods, however, achieve efficient compression with minimal data loss, maintaining reasonable file sizes while preserving key visual content.

3.2 Quality Evaluation

The quality measures in Table 2 indicate how well the different methods upheld the image’s visual and structural fidelity to the original.

- **JPEG/JPEG-LS:** These methods had excellent all-around performance with a Global PSNR of 39.16 dB and an SSIM of 0.952, indicating they retained most overall image details, albeit at slightly lower quality for the region of interest (ROI PSNR 37.04 dB).
- **Block-based PCA:** This method performed worst overall, with a Global PSNR of 14.67 dB and an SSIM of 0.42. The results are consistent with its heavy smoothing, which greatly diminishes important diagnostic details.
- **JPEG 2000 (ROI):** This technique performed best among all, achieving an SSIM of 0.987 and a well-balanced PSNR of 35.58 dB. It effectively retains texture and edge details within the ROI while compressing less vital regions of the image.
- **Proposed Hybrid:** The proposed hybrid approach attained a Global PSNR of 29.44 dB and an ROI PSNR of 34.72 dB, with a High-Quality Index (HQI) of 0.9294. This demonstrates excellent ROI preservation and overall good image reconstruction.

3.3 Comparative Analysis: JPEG 2000 (ROI) vs Proposed Hybrid

Both **JPEG 2000 (ROI)** and the **Proposed Hybrid** techniques aim to balance compression and efficiency of ROI through region-adaptive processing. However, they are different in their implementation and performance trends.

Table 3: Comparison between JPEG 2000 (ROI) and Proposed Hybrid Method

Metric	JPEG 2000 (ROI)	Proposed Hybrid
File Size (KB)	12.98	12.47
Global PSNR (dB)	35.58	29.44
Global SSIM	0.9867	0.8223
ROI PSNR (dB)	32.61	34.72
ROI SSIM	0.9867	0.8223
HQI	0.9657	0.9294

Observations:

- **Compression Efficiency:** The hybrid method results in a file size that is slightly smaller overall. It measures about 12.47 KB, while the other comes in at 12.98 KB. This points to a marginal improvement in storage efficiency.
- **ROI Efficiency:** The hybrid method shows a stronger ROI PSNR score of about 34.72 dB. This points to improved local sharpening and clearer details in those key spots that matter for medical diagnosis.
- **Overall Structural Quality:** JPEG 2000 holds the highest global SSIM score at 0.9867. This indicates smoother transitions overall and fewer artifacts in areas outside the ROI.
- **Balanced Performance (HQI):** JPEG 2000 reaches the highest HQI score of 0.9657. The Hybrid method still manages a competitive HQI of 0.9294. That score highlights solid trade-offs between compression levels and how the image looks to the eye.

References

- [1] Ungureanu, V.-I., Negirla, P., & Korodi, A. (2024). *Image-compression techniques: Classical and “region-of-interest-based” approaches presented in recent papers*. *Sensors*, 24(3), 791. <https://doi.org/10.3390/s24030791>
- [2] Kafashan, M., Hosseini, H., Beygiharchegani, S., Pad, P., & Marvasti, F. (2010, October). New rectangular partitioning methods for lossless binary image compression. In *2010 IEEE 10th International Conference on Signal Processing (ICSP)*. IEEE. <https://doi.org/10.1109/ICOSP.2010.5655736>
- [3] Bairagi, V., & Sapkal, A. M. (2013). ROI-based DICOM image compression for telemedicine. *Indian Academy of Sciences*, 38(1), 123–131. <https://www.ias.ac.in/public/Volumes/sadh/038/01/0123-0131.pdf>
- [4] Kiran, R., & Kamargaonkar, C. (2016, March). Region based medical image compression using block-based PCA. In *2016 International Conference on Computation of Power, Energy, Information and Communication (ICCPEIC)*. IEEE. <https://doi.org/10.1109/ICCPEIC.2016.7557289>

- [5] **Dataset:** *Brain MRI Images for Brain Tumor Detection*. Available at: <https://www.kaggle.com/navoneel/brain-mri-images-for-brain-tumor-detection>